Freight Rate Prediction

Train/Test Validation Approach & December Prediction Chart — Explanation

Part 1 — Approach to Validating and Splitting the Train/Test Data

The provided **train-test.csv** (48,000 labeled loads) is the only data with a known **posted_rate**. Before any model could be trusted, it needed to be split into a portion used for fitting and a portion held back purely for measuring accuracy, in a way that honestly reflects how the model will be used in production: scoring **validation.csv**, a set of 12,000 loads that occurs entirely in the future relative to the training data.

1.1 Why a random split was rejected

train-test.csv spans Jan 1 – Oct 31, 2025. **validation.csv** spans Nov 1 – Dec 31, 2025 — every single row is chronologically after every row the model is trained on. A random 80/20 shuffle of train-test.csv would let the model see rows from every month during training, including the months immediately preceding whichever rows landed in the random test set. That would **overstate** how well the model generalizes to genuinely unseen future months, because it is an easier task than what the model actually faces at scoring time.

1.2 Approach used: time-based holdout split

Fold	Date range	Row count	Role
Development-train	2025-01-01 → 2025-09-15	40,850	Fit imputers, winsorize target, train model
Development-holdout	2025-09-16 → 2025-10-31	7,150	Measure honest out-of-time accuracy

The last ~6 weeks of the labeled data are held out as an internal validation fold. This mirrors the real deployment gap (train on the past, score the future) at a smaller scale, so the holdout metrics are a realistic estimate of performance on **validation.csv** rather than an optimistic, same-period estimate.

1.3 Preventing leakage across the split

- **Imputation statistics** (median weight, median market_index) are computed only from the development-train fold, then applied unchanged to the holdout fold and, later, to validation.csv. The holdout/validation data never influences what value is used to fill a missing cell.
- **Target winsorization** (capping extreme outlier posted_rate values before fitting, at the 0.5th/99.5th percentile) is computed from and applied only to the training fold's target. Evaluation is always against the real, unmodified posted_rate in the holdout fold — the model is never graded against a value it was allowed to see adjusted.
- **No target-derived features**: rate-per-mile, which is derived from posted_rate, is used only for EDA and for one baseline's internal calculation from training data — it is never passed to the model as an input feature.
- **City names dropped as categorical features.** Validation.csv contains 8 pickup/delivery cities (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego) that never appear anywhere in train-test.csv. A model trained with one-hot or target-encoded city features would have no valid encoding for these rows. Latitude/longitude coordinates are used instead, since they are continuous and generalize to any new location without needing to have been seen before.

1.4 Validating the split worked as intended

Before trusting any model comparison, the split itself was sanity-checked:

- Row counts and date boundaries were printed and confirmed to match the intended cutoff (2025-09-16) with no overlap between folds.
- Imputation statistics were logged (e.g. weight_median = 31,485.5 lb, market_median = 1.100) to confirm they were computed only on the training fold.
- A trivial mean-only baseline was scored on the holdout first (MAE $1,174, $R^2 \approx 0$) to confirm the evaluation code produces sane, non-leaked numbers before any real model was evaluated — an artificially high score at this stage would have indicated a leakage bug.

1.5 Final model refit before scoring validation.csv

Once the best-performing model (bagged regression trees) was selected using the holdout, it was **refit from scratch on 100% of train-test.csv** (all 48,000 rows) — not reused from the holdout run — using the same imputation and winsorization logic, before scoring the 12,000 rows of validation.csv. This maximizes the data available to the final production model while keeping the split used to *choose* that model strictly honest.

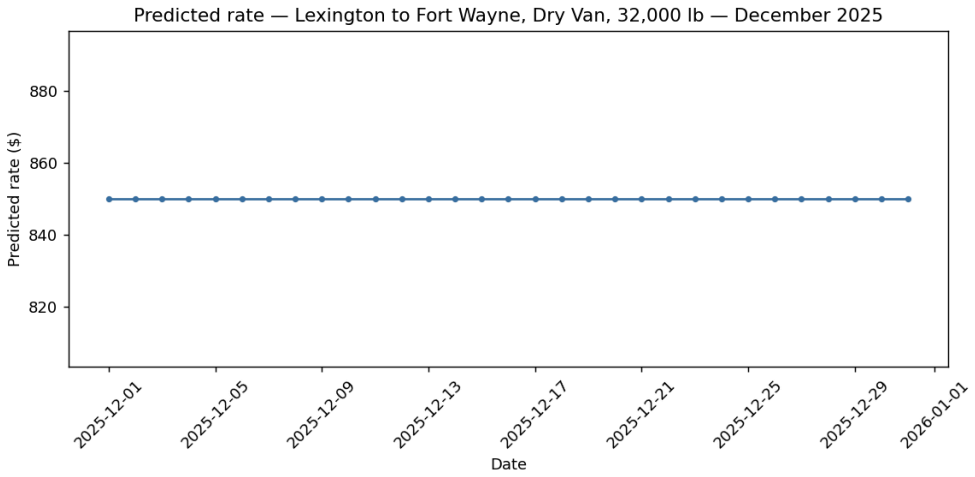
Holdout results that informed model selection:

Model	MAE ($)	RMSE ($)	R2	MAPE (%)
Baseline: mean	1,173.9	1,515.7	-0.00	82.9
Baseline: rate-per-mile × equipment	263.3	666.4	0.807	11.1
Linear Regression (Ridge)	162.7	616.9	0.834	8.7
Regression Tree (single)	149.0	615.3	0.835	6.9
Bagged Trees (n=25) — selected	**135.0**	**610.0**	**0.838**	**6.6**

Bagged trees were selected: best MAE/RMSE/R^2/MAPE on the out-of-time holdout, and able to capture non-linear equipment × distance × region interactions that the linear model misses, while remaining simple enough (25 shallow trees) to stay interpretable via permutation importance.

Part 2 — Why the December Prediction Chart Is Flat

december-chart-inputs.csv asks for a predicted rate for one fixed lane (Lexington → Fort Wayne, Dry Van, 32,000 lb) for every day in December 2025. Running this file through the scoring script (the same final model used to produce validation_predictions.csv) produces a **flat predicted rate of $849.99 on every single day**. This section explains why that is the correct output given the inputs provided, not a bug in the scoring script.



Predicted rate for the fixed lane, by day, across December 2025 (output of the scoring script).

2.1 What the scoring script actually receives

december-chart-inputs.csv provides 7 columns: **pickup, delivery, distance, equipment, weight, date, predicted_rate** (empty, to be filled). Critically, it does **not** provide:

- **pickup_lat / pickup_lon / delivery_lat / delivery_lon** — recovered by the script via a city-name lookup table built from every known coordinate in train-test.csv and validation.csv (both Lexington and Fort Wayne were found).
- **market_index** and **quote_signal** — not present in this file at all. The script imputes both at their training-median value, since no scenario-specific value was supplied for any row.

2.2 Why every day in December looks identical to the model

The model's feature set (see Part 1 / features_models.py) is: distance, pickup/delivery lat & lon, weight, market_index, quote_signal, equipment (one-hot), and month encoded as sin/cos of the calendar month. For every row in december-chart-inputs.csv:

Feature	Value across all 31 rows	Varies day-to-day?
distance, equipment, weight	360 mi, Dry Van, 32,000 lb (fixed by the input file)	No
pickup/delivery lat, lon	Lexington → Fort Wayne coordinates (looked up)	No
market_index, quote_signal	Imputed at training median (not provided per-row)	No
month_sin, month_cos	December for every row	No
date (day-of-month)	Changes 12-01 → 12-31	Yes — but not used as a feature

Every feature the model actually uses is constant across all 31 rows. The only thing that changes row-to-row — the calendar day-of-month — was deliberately **not** engineered into a feature, because it is not a signal the model was trained to use; only day-of-week and month were tested as candidates. A model given 31 identical feature vectors will, correctly, return 31 identical predictions.

2.3 Why day-of-week wasn't added to fix this

During EDA, day-of-week was tested as a candidate feature: mean rate-per-mile by weekday in the training data ranged only from 2.192 to 2.248 (about a 1–2% swing), with a standard deviation of 0.023 across the seven weekdays — noise-level, not a real pricing pattern. Adding it would not have produced a genuine daily trend; it would have manufactured an arbitrary-looking wiggle in the chart that the data doesn't actually support. The flat line is therefore a more honest output than a fabricated one.

2.4 What would be needed for a real day-by-day December forecast

- **Actual daily market_index and quote_signal values** for December (these are the two features most likely to carry real day-to-day market movement, based on permutation importance), rather than a single imputed median for the whole month.
- Confirmation from the business/data owner of what market_index and quote_signal represent, since their real-world meaning was not documented in the provided files.
- If a genuine holiday/seasonal effect exists around late December, it was not detectable in the Jan–Oct training window (which contains no December data at all) — this is a data-coverage gap, not something the model or script can infer.
